
Progress Report for Conditioned Jazz Solo Generation Using Transformer Model and Chord Embeddings

Curtis Hei-yin Ho

1011059845, curti.ho@mail.utoronto.ca

<https://colab.research.google.com/drive/10ZLgXrWZsanGnocS-YUYOdTu-xHTxJBV?usp=sharing>

Project Description

The objective of the project is to develop a model that generates a line of musical improvisations according to a given chord progression or sequence. This is accomplished by training a neural network on Musical Instrument Digital Interface (MIDI) files of jazz transcriptions, allowing the model to learn complex patterns and stylistic nuances, ultimately generating music that emulates the likes of jazz improvisation.

Music improvisation is a complex art form. Despite theoretical musical guidelines, as one may learn in music school, mastery of improvisation can be both elusive and intuitive. While some learners excel with minimal formal theory, others may struggle despite diligent study. Hence, I aspire to explore the balance as to the extent improvisational skill can be codified by rules, and how much it can be acquired simply by exposure.

The advent of deep learning has made it easy for computers to recognize complex patterns in data, and break them down into simpler features. By using deep learning to model jazz improvisation, this project could reveal how much of the improvisational process can be learned from patterns in the music itself, without explicit human-imposed restrictions or guidance (1). The project also carries the potential to inform human learning in creating music.

Data Processing

To begin, I collected over 800 jazz MIDI files from two publicly available databases (mididb.com and midkar.com). Each MIDI file contains one or more instrument tracks. Therefore, I process the MIDI files with a library, `pretty_mid`, and select the non-drum instrument track with the greatest number of notes as the melody track. For each note, I extracted the pitch, the quantized onset time (when the note starts playing) and the quantized duration (how long the note plays for). These values are quantized (rounded to the nearest 0.25 seconds), and range from 0.25 to 2.0 for simplicity. If a gap between notes was detected, a rest event with the same onset and duration is added. With the melody extraction completed, the integral part of the project is chord extraction.

Chord labels are assigned using a windowed approach (2). Over the entirety of a MIDI file, a sliding window (0.5 seconds long with a 0.5 second step) collects all notes currently playing in each window, computes the set of unique pitch classes, and labels the window with a string like "0-4-7" (which would be a C major triad). Windows that do not have at least two active notes are labeled as an absent chord. After constructing this timeline of chord labels, each melody event is paired with its closest chord label by matching onset times.

Then, the dataset is cleaned by filtering out melody events with absent chords. Songs with a significant number of ambiguous chord labels are discarded. This process refines the dataset, resulting in a large collection of melody events, each paired to a clear harmonic context.

Here is an example of one note in an extracted melody, with the accompanying chord:

(0, 4.6000000000000005, 0.8999999999999995, 0, '2-5-10') A melody would be a list containing them.

Upon completion of the data extraction, I have collected a total of 161099 notes/rests and 161099 recognized chords.

A major challenge I faced during the data cleaning and preparation was the extraction of accurate chord labels from raw MIDI files. Initially, I tried using the music21 library, which provides an advanced built-in tool for chord identification and other music analyses. Despite its potential for higher accuracy in chord-labeling, music21 proved too computationally heavy for our extensive dataset. Given the large number of MIDI files and the potentially lengthy duration of each file, the processing time required by music21 was not practical. Therefore, I opted for a custom sliding window approach using the pretty_midi library. This method computes pitch-class sets within time windows by taking all sounding notes and forming a set, sacrificing musical nuance and theoretical accuracy. Despite being quicker, it may mislabel complex chords, but more importantly, it simply lumps all notes into a single pitch-class set, even if those notes are passing tones that do not define the underlying harmonic chord.

Figure 1 shows the data processing architecture of the project.

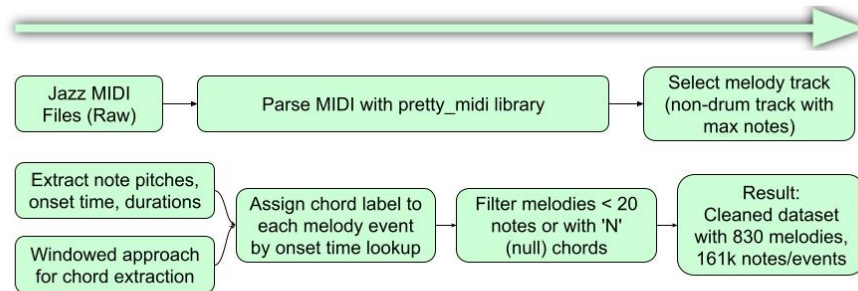


Figure 1: Data Processing Overview

After processing, I randomly shuffled and split the filtered dataset into training (80%), validation (10%), and test (10%) sets. This creates a test set that contains sequences the model has never seen during training or validation.

Baseline Model

As a baseline model for reference purposes, a simple Long Short-Term Memory (LSTM) model, trained on the same processed MIDI data, will be used to generate improvisational note sequences while accounting for chord context learned in the training data.

The LSTM stack includes one layer, comprising 128 hidden units, with gating mechanisms to selectively retain and forget musical context across time. The LSTM processes the concatenated input sequence and produces a sequence of hidden states; at each prediction step, the hidden state corresponding to the last time step is extracted. This terminal hidden state carries an encoded summary of the entire input phrase and is passed to two parallel fully connected (dense) output heads. One head predicts the next melody pitch, and the other predicts the next duration class, both trained using cross-entropy loss.

While the LSTM effectively learns local melodic and harmonic relationships and requires minimal hyperparameter tuning, its core limitation lies in its difficulty in modeling the long-range dependencies and complex stylistic structure inherent to jazz improvisation. This often results in generated melodies that lack the extended coherence achieved by attention-based models (3).

Primary Model

The primary model is a Transformer-based neural network with 6 encoder and 6 decoder layers, leveraging self-attention mechanisms with 8 attention heads to effectively capture both short and long-range dependencies in musical sequences. Learned positional embeddings are added to the input embeddings at each time step, and causal masking ensures the model can only attend to previous time steps during generation.

Input features are encoded through three separate embedding layers: pitch (64 dimensions), chord context (32 dimensions), and duration (32 dimensions), which are concatenated to form a 128-dimensional representation. The feedforward layers use a hidden dimension of 256. Crucially, the pitch information is encoded using a hybrid interval-absolute scheme: the first note in a sequence and notes following rests are encoded as absolute MIDI pitches, while consecutive sounding notes are encoded as intervals (pitch differences) from the previous note (4). This significantly simplifies the learning task compared to predicting absolute MIDI pitch values.

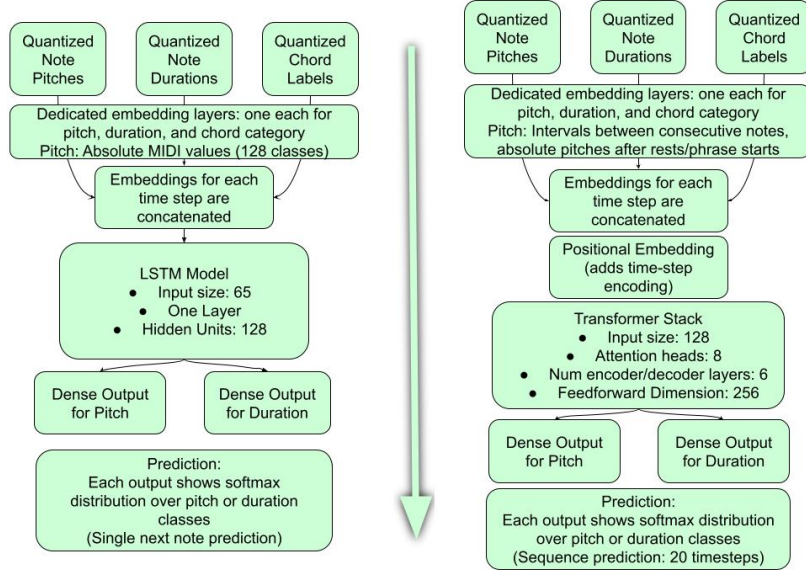


Figure 2: Block diagrams comparing the baseline LSTM model (left) and the primary Transformer model (right) for melody generation

Outcomes and Next Steps

The primary model, using relative interval encoding for a transformer model, delivered slightly better performance (Test Pitch Accuracy: 0.2915; Test Duration Accuracy: 0.8305) compared to the baseline LSTM (test pitch accuracy: 0.2304; Test Duration Accuracy: 0.8266). However, accuracy is a measure that is difficult to assess in this application. Appendix A visualizes the performances of the two models through train and validation losses and accuracies. Although the losses show a healthy model, the subjective sound test may be another valuable evaluation, as it revealed that the Transformer model generates melodies with greater stylistic nuance and phrasing, whereas the LSTM shows signs of melodic discontinuity (jumpiness). Future work will therefore involve further architecture refinement but also more generation parameter tuning, like adjusting the logit penalty for the rest token and adjusting the sampling temperature to achieve musically coherent and stylistically satisfying output.

Appendix A

The graphs show that the transformer has a slightly stronger ability to learn.

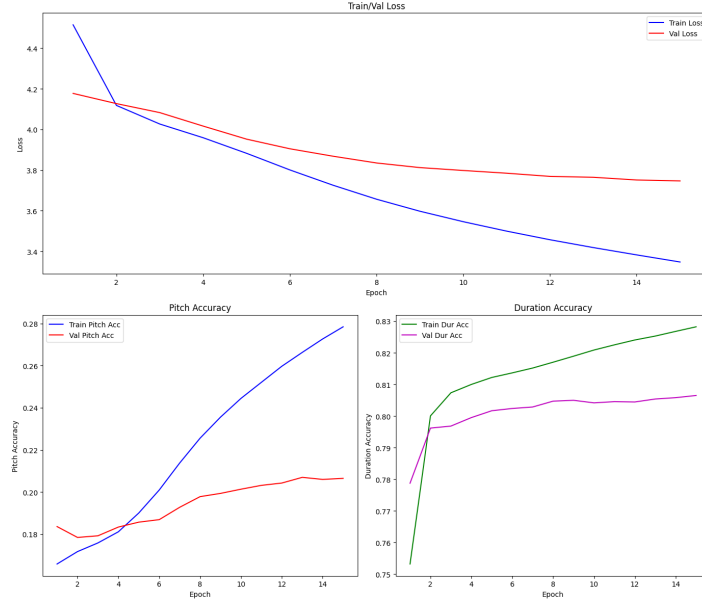


Figure 3: The training and validation metrics for the Transformer model. The top panel shows the convergence of the Categorical Cross-Entropy Loss over the epochs. The bottom panel demonstrates the progression of the Pitch (Interval) and Duration Classification Accuracy.

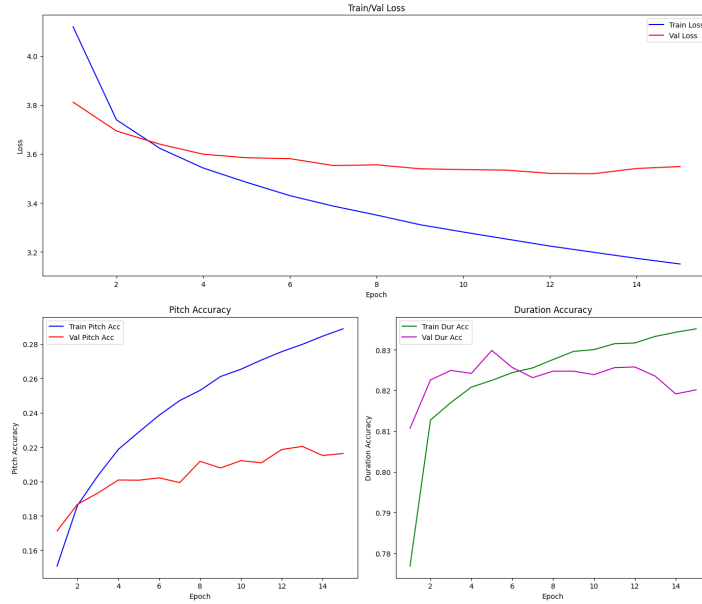


Figure 4: The training and validation metrics for the Long Short-Term Memory (LSTM) baseline model. The top panel displays the convergence of the Categorical Cross-Entropy Loss over the epochs. The bottom panel shows the progression of the Pitch and Duration Classification Accuracy. Note that the pitch accuracy ceiling is low due to the challenge of predicting absolute MIDI pitch values.

References

- [1] A. Nayebi and M. Vitelli, “Gruv: Algorithmic music generation using recurrent neural networks,” 2015. CS224D project report.
- [2] L. Oudre, Y. Grenier, and C. Févotte, “Chord recognition by fitting rescaled chroma vectors to chord templates,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 19, no. 2, pp. 366–377, 2011.
- [3] R. Abbou, “Deepclassic: Music generation with neural networks,” tech. rep., Stanford University, 2020. CS224N Custom Project Report.
- [4] D.-V.-T. Le, L. Bigo, and M. Keller, “Evaluating interval-based tokenization for pitch representation in symbolic music analysis,” *arXiv preprint arXiv:2501.04630*, 2025.